

Compilers

Problem Set 4 — Lexical Analysis

Time. About 2 hours. **Total:** 100 points.

Tools. You may use an LLM, flex, a programming language, anything. You are responsible for the correctness of everything you hand in. Several problems ask for exact counts or exact output, and a plausible-looking answer that is off by one earns nothing.

Verification note (1 point per problem). End every problem with a note of at most two lines saying *how you checked your answer independently of the tool that produced it* (a hand trace, a script you wrote, flex output, a counter-example you tried...). “The LLM said so” earns no point.

Conventions used throughout

- Regular expressions use *flex* syntax: `.` is any character except newline, `[^x]` is a complement class (it *does* contain newline), `"..."` is a literal string, `\.` escapes a special character, and `*` `+` `?` have their usual meaning.
- A scanner built from an ordered list of rules uses the *longest match* principle; among rules matching the same longest lexeme, the rule listed *first* wins. If nothing matches, the scanner fails (unless a catch-all rule exists).
- **Thompson’s construction** is the one on the slide “from regular expression to NFA” (Dragon book): $N(\varepsilon)$ and $N(a)$ have 2 states; $N(s|t)$ and $N(s^*)$ add a new initial and a new final state; in $N(st)$ the final state of $N(s)$ is *merged* with the initial state of $N(t)$ (no new state, no ε -edge).

Problem 1 — Designing and criticizing regexes

(16 pts, $\approx$ 20 min)

- (a) (4) Write *one* flex regular expression that matches exactly the C-style block comments: it starts with `/*`, ends at the *first* `*/` after that, may span several lines, and cannot be nested. For each string below, say whether your expression matches it *entirely*:

```
1  /**/
2  /***/
3  /*/
4  /* a * b **/
5  /* a */ b */
6  a two-line string: /* 11 newline 12 */
```

- (b) **(3)** A student proposes the rule `"/*".**"/` for comments (together with an identifier rule and a rule skipping blanks). (i) Which tokens does the resulting scanner produce for `a /* c1 */ b /* c2 */ c`? (ii) What happens to a comment spanning two lines? (iii) Explain (i) using the longest match principle, and explain why moving this rule earlier or later in the rule list cannot fix it.
- (c) **(3)** Give the *minimal* DFA of your expression from (a), over the three character classes `{/, *, other}`. How many states does it have (counting the dead state), and what does each state “remember”? Use it to explain why `/*/` is not a comment.
- (d) **(5)** Numeric literals: an integer part that is either 0 or has no leading zero; an optional fraction (a point followed by *at least one* digit); an optional exponent (`e` or `E`, optional sign, at least one digit) which may follow an integer part directly. Write a regular expression for these literals. Then, using it as rule `NUM`, plus rules `ID` (`[a-zA-Z_][a-zA-Z0-9_]*`), `DOT`, `PLUS`, `MINUS` and a rule skipping blanks, give the token stream for

007 1.e5 1e- 12e3.4

and explain in one sentence why `007` does not produce a single token.

Verification note (1).

Problem 2 — Thompson’s construction: size and structure (18 pts, ≈20 min)

Let $r = (a|b)^*abb$.

- (a) **(5)** Build $N(r)$ with the construction above. Give the complete list of states and edges (any consistent numbering), the initial state and the final state. How many states and how many edges does it have?
- (b) **(3)** The slide lists three properties of $N(r)$: (1) at most twice as many states as operators plus operands, (2) one initial and one final state, with no incoming edge to the initial state and no outgoing edge from the final state, (3) every non-final state has either one outgoing edge or two outgoing edges, both on ε . Verify each one on your automaton. How many states have two outgoing ε -edges?
- (c) **(4)** Consider regular expressions built only from single symbols (or ε), $|$, $*$ and concatenation. Let n_{sym} , n_{alt} , n_{star} , n_{cat} be the number of symbol leaves, alternations, stars and concatenations. Derive the exact number of states of $N(r)$ in terms of these numbers, then express it in the form

$$\#states = 2(\#operators + \#operands) - (...).$$

Conclude property (1), and say exactly *when* the bound is tight.

- (d) **(5)** The slides say that simulating an NFA with $|Q|$ states on a string of length $|S|$ takes $O(|S||Q|^2)$ in the worst case. Show that for automata produced by Thompson’s construction one simulation step (the ε -closure of the move) costs only $O(|Q|)$, hence $O(|S||Q|)$ overall, and pinpoint where the $|Q|^2$ of the general bound comes from. Then state what a DFA buys on top of that.

Verification note (1).

Problem 3 — Maximal munch, pushback, a quadratic trap (20 pts, ≈ 25 min)

A scanner has the following rules, listed in priority order:

IF	if	DOT	\.
ID	[a-z][a-z0-9]*	PLUS	\+
FLT	[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?	MINUS	-
NUM	[0-9]+	WS	[]+ (skipped)
LE	<=	ERR	. (catch-all)
LT	<		
EQ	=		

and it scans the input

```
if1<=3.14e+x 2.e5 iffy=1e-;
```

Reads. The scanner reads characters one at a time from a buffer. When every automaton is stuck it stops, pushes back everything read after the end of the chosen lexeme, and restarts right after the lexeme. A pushed-back character is read again later and counts *again*. The character on which the last automaton gets stuck counts as read. Nothing is counted at end of file. Skipped blanks also cost reads.

- (a) (4) Give the token stream (omit WS).
- (b) (4) How many character reads does the scanner perform in total? Which token causes the largest pushback, and how many characters does it push back?
- (c) (3) Over *all possible inputs*, what is the maximum number of characters that a single token of this scanner can push back? Justify (an example achieving it, and an argument that it cannot be exceeded).
- (d) (4) Now take only two rules, A: **a** and B: **a*b**, and the input a^n (no b at all). How many reads does the scanner perform, exactly, as a function of n ? What is the complexity, and how much can one token push back on this rule set?
- (e) (4) Describe a modification of the scanner (you may store information between tokens) making it linear on this family *without changing the tokens it outputs*. State precisely what you record and when, prove that it never changes the output, and state the resulting complexity.

Verification note (1).

Problem 4 — When DFAs must be exponential (16 pts, ≈ 15 min)

For $k \geq 1$ let L_k be the set of strings over $\{a, b\}$ whose k -th symbol *from the end* is a , i.e. $L_k = (a|b)^* a (a|b)^{k-1}$.

- (a) **(3)** Give an NFA for L_3 with only 4 states and no ε -edges (transition table).
- (b) **(4)** Run the subset construction on it. List every reachable DFA state as a set of NFA states, its transitions, and which states are accepting. How many states do you get?
- (c) **(6)** Prove that every DFA for L_k has at least 2^k states. (Hint: find 2^k strings that are pairwise distinguishable, and for two such strings exhibit the distinguishing suffix.) Also say how many states Thompson's construction gives for the regular expression of L_k , as a function of k , and what this shows about the slide's claim on the size of DFAs.
- (d) **(2)** The slides note that minimization does not remove this blow-up. In one or two sentences, what does that tell you about choosing between an NFA scanner and a DFA scanner for $k = 25$?

Verification note (1).

Problem 5 — Indentation and the limits of automata (14 pts, ≈ 15 min)

A Python-like lexer treats indentation as follows. It keeps a stack of indentation widths, initially [0]. Blank lines and comment-only lines are ignored. For every other line, with w leading spaces: if $w > \text{top}$ of stack, push w and emit **INDENT**; if $w < \text{top}$, pop and emit one **DEDENT** per pop until $\text{top} \leq w$, and if $\text{top} \neq w$ then report **ERROR** (inconsistent dedent) and stop. At end of file emit one **DEDENT** for each remaining entry above 0.

- (a) **(4)** Give the **INDENT**/**DEDENT** tokens (with the line number that triggers each) for the following file, and say what happens for the second one:

File 1 (line: indent, text)	File 2
1: 0, def f(x):	1: 0, if x:
2: 4, if x:	2: 8, a
3: 8, return 1	3: 4, b
4: (blank)	
5: 4, # done	
6: 4, return 2	
7: 0, print (f(1))	

- (b) **(3)** A file has lines with indentation 0, 2, 4, 6, 0 (each of the first four ends with a colon, the last does not). Which tokens are emitted at line 5? Show that the total number of **DEDENT**s over any file is at most the number of lines, and conclude the amortized cost per line of the stack handling.
- (c) **(4)** Prove that no finite automaton (more precisely: no finite-state machine that outputs tokens as it reads lines) can implement this lexer. (Use lines of the form “ i spaces followed by a colon” as a family of prefixes and a pigeonhole argument.) Relate it to what the lecture says about the power of regular languages.
- (d) **(2)** The slides mention that in Fortran, `D0 5 I = 1,25` is a loop header while `D05I = 1.25` is an assignment, and blanks are irrelevant. Why can longest match on the character stream alone not decide whether `D0` is a keyword, and what kind of mechanism does a scanner need?

Problem 6 — flex: predicting and writing scanners

(16 pts, ≈25 min)

- (a) (4) Consider this flex specification (yy`leng` is the length of the lexeme):

```
%%
ab*      printf("[1:%s]", yytext);
a*b      printf("[2:%s]", yytext);
a        printf("[3]");
b+       printf("[4:%d]", yyleng);
.|\\n    printf("[.]");
%%
```

Predict the exact output on the input `aabbbaab` followed by a newline and then `abbab`. Is rule 3 ever executed? Justify, and say what flex reports about it.

- (b) (7) Write a flex specification for the following “Tiny” language and run it on the test file below.

- Keywords `if`, `else`, `while`; identifiers `[A-Za-z_][A-Za-z0-9_]*`; integers (0 or no leading zero) and hexadecimal integers (`0x` or `0X` followed by at least one hex digit); string literals in double quotes with backslash escapes and no raw newline inside; operators `==`, `!=`, `<=`, `<`, `=`, `/`; punctuation `( ) { } ; .`
- Line comments `//` to end of line and non-nested block comments (which may span lines) are skipped, as is whitespace.
- Every token is printed as `line:col KIND lexeme`, with 1-based line and column of its first character, correct across newlines *including newlines inside skipped comments*. Kinds: `KEYWORD`, `ID`, `INT`, `HEX`, `STRING`, `OP`, `PUNCT`.
- Any other character prints `line:col error: unexpected 'c'` and scanning continues.

Test file (four lines, the last one has no closing quote):

```
if (x0 == 0x1F) { y = 007; } // check
else while (iffy != "a\"b") /* multi
line */ z = 0x; a / b @
"open
```

Hand in your specification and its exact output.

- (c) (4) Without running them, predict the effect of two mutations of your specification, then check with flex: (i) the identifier rule is moved *before* the keyword rule; (ii) the block-comment rule is replaced by `"/*.*/"`, and the input is `a /* c1 */ b /* c2 */ c` (also say what happens to the multi-line comment in the test file).